

Vòng lặp cơ bản (Basic Loops)

Nội dung

Trong các bài trước, mỗi lệnh trong chương trình chỉ được thực hiện đúng một lần. Nhưng trong thực tế, ta thường cần lặp lại một hành động nhiều lần: in ra 100 số, tính tổng 1000 phần tử, đọc dữ liệu cho đến khi hết,... Đó chính là lúc ta cần vòng lặp.

1. Tại sao cần vòng lặp?
2. Vòng lặp for
 - Cú pháp
 - Ví dụ 1: In các số từ 1 đến n
 - Ví dụ 2: Đếm ngược từ n về 1
 - Ví dụ 3: In các số chẵn từ 2 đến n
3. Vòng lặp while
 - Cú pháp
 - Ví dụ 4: Đọc số cho đến khi gặp số 0 (sentinel value)
4. Mẫu tích lũy (Accumulator Pattern)
 - Tính tổng
 - Đếm số phần tử thỏa điều kiện
 - Tính tổng các số lẻ
5. Vòng lặp với điều kiện lọc
 - Ví dụ: Tìm các số chia hết cho 3 trong đoạn $[a, b]$
 - Ví dụ: Tìm các ước của n
6. Xử lý chữ số bằng while
 - Ví dụ: Đếm số chữ số và tính tổng chữ số
7. break và continue
 - break — Thoát khỏi vòng lặp ngay lập tức
 - continue — Bỏ qua phần còn lại, sang lần lặp tiếp theo
8. Bài toán mẫu: Số hoàn hảo
9. Bài toán mẫu: Phân tích thừa số nguyên tố
10. Mẹo và lỗi thường gặp
 - Lỗi lệch 1 (Off-by-one error)
 - Lặp vô hạn (Infinite loop)
 - Quên khởi tạo biến tích lũy
 - Nhầm lẫn giữa for và while
 - Kiểu dữ liệu tràn số

1. Tại sao cần vòng lặp?

Giả sử ta muốn in các số từ 1 đến 5. Không có vòng lặp, ta phải viết:

C++

```
cout << 1 << endl;  
cout << 2 << endl;  
cout << 3 << endl;  
cout << 4 << endl;  
cout << 5 << endl;
```

Python

```
print(1)  
print(2)  
print(3)  
print(4)  
print(5)
```

Nếu muốn in từ 1 đến 1000 thì sao? Ta không thể viết 1000 dòng! Vòng lặp giúp ta viết một lần nhưng chạy nhiều lần:

C++

```
for (int i = 1; i <= 1000; i++) {  
    cout << i << endl;  
}
```

Python

```
for i in range(1, 1001):  
    print(i)
```

Chỉ 3 dòng code thay vì 1000 dòng!

2. Vòng lặp for

Vòng lặp for dùng khi ta biết trước số lần lặp (hoặc phạm vi lặp).

Cú pháp

C++

```
for (khởi_tạo; điều_kiện; cập_nhật) {  
    // thân vòng lặp - chạy khi điều_kiện đúng  
}
```

Giải thích:

- Khởi tạo: chạy 1 lần duy nhất ở đầu (ví dụ `int i = 0`)
- Điều kiện: kiểm tra trước mỗi lần lặp; nếu đúng thì chạy thân, nếu sai thì dừng
- Cập nhật: chạy sau mỗi lần lặp (ví dụ `i++`)

Python

```
for biến in range(bắt_đầu, kết_thúc):  
    # thân vòng lặp
```

Giải thích:

- `range(n)` tạo dãy số $0, 1, 2, \dots, n - 1$
- `range(a, b)` tạo dãy số $a, a + 1, \dots, b - 1$ (không bao gồm b)
- `range(a, b, step)` tạo dãy với bước nhảy `step`

Ví dụ 1: In các số từ 1 đến n

C++

```
#include <iostream>  
using namespace std;  
int main() {  
    int n;  
    cin >> n;  
    for (int i = 1; i <= n; i++) {  
        cout << i << " ";  
    }  
    return 0;  
}
```

Python

```
n = int(input())  
for i in range(1, n + 1):  
    print(i, end=" ")
```

Chạy tay với $n = 5$:

BƯỚC	I	ĐIỀU KIỆN	IN RA
1	1	$1 \leq 5$ ✓	1
2	2	$2 \leq 5$ ✓	2
3	3	$3 \leq 5$ ✓	3
4	4	$4 \leq 5$ ✓	4
5	5	$5 \leq 5$ ✓	5
6	6	$6 \leq 5$ × (dừng)	

Ví dụ 2: Đếm ngược từ n về 1

C++

```
for (int i = n; i >= 1; i--) {  
    cout << i << " ";  
}
```

Python

```
for i in range(n, 0, -1):  
    print(i, end=" ")
```

Ví dụ 3: In các số chẵn từ 2 đến n

C++

```
// Cách 1: dùng bước nhảy 2  
for (int i = 2; i <= n; i += 2) {  
    cout << i << " ";  
}  
  
// Cách 2: dùng if kiểm tra  
for (int i = 1; i <= n; i++) {  
    if (i % 2 == 0) {  
        cout << i << " ";  
    }  
}
```

Python

```
# Cách 1: dùng bước nhảy 2  
for i in range(2, n + 1, 2):  
    print(i, end=" ")  
  
# Cách 2: dùng if kiểm tra  
for i in range(1, n + 1):  
    if i % 2 == 0:  
        print(i, end=" ")
```

3. Vòng lặp while

Vòng lặp while dùng khi ta không biết trước số lần lặp — lặp cho đến khi một điều kiện nào đó không còn đúng.

Cú pháp

C++

```
while (điều_kiện) {
    // thân vòng lặp - chạy khi điều_kiện đúng
}
```

Python

```
while điều_kiện:
    # thân vòng lặp
```

Lưu ý quan trọng: Trong thân vòng lặp, phải có lệnh làm thay đổi điều kiện, nếu không sẽ bị lặp vô hạn!

Ví dụ 4: Đọc số cho đến khi gặp số 0 (sentinel value)

Đề bài: Nhập các số nguyên cho đến khi gặp số 0, in ra tổng các số đã nhập.

C++

```
#include <iostream>
using namespace std;
int main() {
    int x, tong = 0;
    cin >> x;
    while (x != 0) {
        tong += x;
        cin >> x;
    }
    cout << tong << endl;
    return 0;
}
```

Python

```
tong = 0
x = int(input())
while x != 0:
    tong += x
    x = int(input())
print(tong)
```

Chạy tay với input 3 7 -2 0:

BƯỚC	X	ĐIỀU KIỆN	TONG
Đầu	3	$3 \neq 0 \checkmark$	$0 + 3 = 3$
2	7	$7 \neq 0 \checkmark$	$3 + 7 = 10$
3	-2	$-2 \neq 0 \checkmark$	$10 + (-2) = 8$
4	0	$0 \neq 0 \times$	(dừng, in 8)

4. Mẫu tích lũy (Accumulator Pattern)

Đây là kỹ thuật cực kỳ phổ biến: dùng một biến để tích lũy kết quả qua mỗi lần lặp.

Trước tiên, hãy biết về toán tử gán kết hợp — cách viết tắt rất hay dùng:

VIẾT TẮT	TƯƠNG ĐƯƠNG	Ý NGHĨA
<code>a += b</code>	<code>a = a + b</code>	Cộng thêm b vào a
<code>a -= b</code>	<code>a = a - b</code>	Trừ bớt b khỏi a
<code>a *= b</code>	<code>a = a * b</code>	Nhân a với b
<code>a /= b</code>	<code>a = a / b</code>	Chia a cho b
<code>a %= b</code>	<code>a = a % b</code>	Lấy phần dư a chia b

Cú pháp này giống nhau trong cả C++ và Python. Trong C++, còn có `a++` (tăng 1) và `a--` (giảm 1).

Tính tổng

C++

```
int tong = 0; // khởi tạo tổng = 0
for (int i = 1; i <= n; i++) {
    tong += i; // cộng dồn
}
cout << tong << endl;
```

Python

```
tong = 0
for i in range(1, n + 1):
    tong += i
print(tong)
```

Đếm số phần tử thỏa điều kiện

Ví dụ: Đếm có bao nhiêu số lẻ trong các số từ 1 đến n .

C++

```
int dem = 0; // khởi tạo bộ đếm = 0
for (int i = 1; i <= n; i++) {
    if (i % 2 != 0) {
        dem++; // tăng bộ đếm
    }
}
cout << dem << endl;
```

Python

```
dem = 0
for i in range(1, n + 1):
    if i % 2 != 0:
        dem += 1
print(dem)
```

Tính tổng các số lẻ

Kết hợp tích lũy và điều kiện: chỉ cộng dồn khi phần tử thỏa mãn.

C++

```
int tong_le = 0;
for (int i = 1; i <= n; i++) {
    if (i % 2 != 0) {
        tong_le += i;
    }
}
cout << tong_le << endl;
```

Python

```
tong_le = 0
for i in range(1, n + 1):
    if i % 2 != 0:
        tong_le += i
print(tong_le)
```

5. Vòng lặp với điều kiện lọc

Nhiều bài toán yêu cầu ta duyệt qua một phạm vi và chỉ xử lý các phần tử thỏa mãn điều kiện nào đó.

Ví dụ: Tìm các số chia hết cho 3 trong đoạn [a, b]

C++

```
for (int i = a; i <= b; i++) {
    if (i % 3 == 0) {
        cout << i << " ";
    }
}
```

Python

```
for i in range(a, b + 1):
    if i % 3 == 0:
        print(i, end=" ")
```

Ví dụ: Tìm các ước của n

C++

```
for (int i = 1; i <= n; i++) {
    if (n % i == 0) {
        cout << i << " ";
    }
}
```

Python

```
for i in range(1, n + 1):
    if n % i == 0:
        print(i, end=" ")
```

6. Xử lý chữ số bằng while

Một ứng dụng kinh điển của `while` là tách từng chữ số của một số nguyên.

Ý tưởng:

- Chữ số cuối cùng = $n \pmod{10}$
- Bỏ chữ số cuối = $n/10$ (chia nguyên)
- Lặp cho đến khi $n == 0$

Ví dụ: Đếm số chữ số và tính tổng chữ số

C++

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    int so_chu_so = 0;
    int tong_chu_so = 0;
    int tmp = n;
    if (tmp == 0) {
        so_chu_so = 1; // số 0 có 1 chữ số
    }
    while (tmp > 0) {
        int chu_so = tmp % 10;
        tong_chu_so += chu_so;
        so_chu_so++;
        tmp /= 10;
    }
    cout << "So chu so: " << so_chu_so << endl;
    cout << "Tong chu so: " << tong_chu_so << endl;
```



```

    return 0;
}

```

Python

```

n = int(input())
so_chu_so = 0
tong_chu_so = 0
tmp = n
if tmp == 0:
    so_chu_so = 1
while tmp > 0:
    chu_so = tmp % 10
    tong_chu_so += chu_so
    so_chu_so += 1
    tmp //= 10
print("So chu so:", so_chu_so)
print("Tong chu so:", tong_chu_so)

```

Chạy tay với $n = 1234$:

BƯỚC	TMP	CHU_SO (TMP%10)	TONG_CHU_SO	SO_CHU_SO
1	1234	4	4	1
2	123	3	7	2
3	12	2	9	3
4	1	1	10	4
5	0	(dừng)	10	4

7. break và continue

Hai lệnh đặc biệt giúp kiểm soát luồng bên trong vòng lặp.

break — Thoát khỏi vòng lặp ngay lập tức

C++

```

// Tìm bội số của 7 đầu tiên lớn hơn 100
for (int i = 101; i <= 1000; i++) {
    if (i % 7 == 0) {
        cout << "Tim thay: " << i << endl;
        break; // thoát ngay, không cần kiểm tra tiếp
    }
}
// Kết quả: Tim thay: 105

```

Python

```
for i in range(101, 1001):
    if i % 7 == 0:
        print("Tim thay:", i)
        break
# Kết quả: Tim thay: 105
```

continue — Bỏ qua phần còn lại, sang lần lặp tiếp theo

C++

```
// In tất cả số từ 1 đến 10, bỏ qua bội của 3
for (int i = 1; i <= 10; i++) {
    if (i % 3 == 0) {
        continue; // bỏ qua, nhảy sang i tiếp theo
    }
    cout << i << " ";
}
// Kết quả: 1 2 4 5 7 8 10
```

Python

```
for i in range(1, 11):
    if i % 3 == 0:
        continue
    print(i, end=" ")
# Kết quả: 1 2 4 5 7 8 10
```

Lưu ý: `break` và `continue` chỉ ảnh hưởng đến vòng lặp gần nhất chứa chúng.

8. Bài toán mẫu: Số hoàn hảo

Đề bài: Kiểm tra xem số nguyên dương n có phải là số hoàn hảo không. Số hoàn hảo là số bằng tổng các ước dương của nó (không tính chính nó). Ví dụ: $6 = 1 + 2 + 3$.

Phân tích:

1. Duyệt tất cả số từ 1 đến $n - 1$
2. Nếu i là ước của n (tức $n \% i == 0$), cộng vào tổng
3. So sánh tổng với n

C++

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    int tong = 0;
```

```

    for (int i = 1; i < n; i++) {
        if (n % i == 0) {
            tong += i;
        }
    }
    if (tong == n) {
        cout << "YES" << endl;
    } else {
        cout << "NO" << endl;
    }
    return 0;
}

```

Python

```

n = int(input())
tong = 0
for i in range(1, n):
    if n % i == 0:
        tong += i
if tong == n:
    print("YES")
else:
    print("NO")

```

9. Bài toán mẫu: Phân tích thừa số nguyên tố

Đề bài: Cho số nguyên dương n , phân tích n thành tích các thừa số nguyên tố. Ví dụ: $12 = 2 \times 2 \times 3$, hay viết gọn $12 = 2^2 \times 3$.

Thuật toán:

1. Duyệt i từ 2 trở đi
2. Khi n chia hết cho i : chia n cho i (lặp lại cho đến khi không chia hết nữa)
3. Sang i tiếp theo
4. Dừng khi $n = 1$

C++

```

#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    for (int i = 2; i * i <= n; i++) {
        while (n % i == 0) {
            cout << i << " ";
            n /= i;
        }
    }
}

```

```

    }
}
if (n > 1) {
    cout << n; // n còn lại là thừa số nguyên tố
}
cout << endl;
return 0;
}

```

Python

```

n = int(input())
i = 2
while i * i <= n:
    while n % i == 0:
        print(i, end=" ")
        n //= i
    i += 1
if n > 1:
    print(n, end="")
print()

```

Tại sao chỉ cần duyệt đến $\sqrt{n}$ (tức $i * i \leq n$)? Vì nếu có thừa số nguyên tố $p > \sqrt{n}$, thì chỉ có duy nhất một thừa số như vậy (đó chính là n còn lại sau khi chia hết các thừa số nhỏ). Dòng `if (n > 1)` ở cuối xử lý trường hợp đó.

10. Mẹo và lỗi thường gặp

Lỗi lệch 1 (Off-by-one error)

Đây là lỗi phổ biến nhất khi viết vòng lặp!

Muốn lặp n lần:

- C++: `for (int i = 0; i < n; i++)` ✓ $(0, 1, \dots, n-1)$
- C++: `for (int i = 1; i <= n; i++)` ✓ $(1, 2, \dots, n)$
- C++: `for (int i = 0; i <= n; i++)` × (lặp $n+1$ lần!)
- Python: `for i in range(n)` ✓ $(0, 1, \dots, n-1)$
- Python: `for i in range(1, n+1)` ✓ $(1, 2, \dots, n)$
- Python: `for i in range(1, n)` × (chỉ lặp $n-1$ lần!)

Mẹo: Luôn chạy tay với giá trị nhỏ ($n = 3$ hoặc $n = 1$) để kiểm tra biên.

Lặp vô hạn (Infinite loop)

C++

```
while (x > 0) {  
    cout << x;  
    // Quên x-- -> lặp mãi mãi!  
}
```

Mẹo: Khi viết `while`, hãy tự hỏi: "Điều gì trong thân vòng lặp sẽ làm điều kiện trở thành sai?" Nếu không trả lời được, bạn đang có bug.

Quên khởi tạo biến tích lũy

C++

```
int tong; // Chưa khởi tạo! Có thể chứa giá trị rác  
for (int i = 1; i <= n; i++) {  
    tong += i;  
}  
// Sửa: int tong = 0;
```

Python

```
# Python sẽ báo lỗi nếu dùng biến chưa gán  
# tong += i -> NameError: name 'tong' is not defined  
# Sửa: khởi tạo tong = 0 trước vòng lặp
```

Nhầm lẫn giữa for và while

- Dùng `for` khi biết trước số lần lặp hoặc phạm vi
- Dùng `while` khi lặp đến khi một điều kiện thỏa mãn

Kiểu dữ liệu tràn số

Khi tính tổng nhiều số, kết quả có thể vượt quá giới hạn của `int` (khoảng 2×10^9). Trong trường hợp đó, dùng `long long` (C++) hoặc Python tự xử lý số lớn.

C++

```
long long tong = 0; // dùng long long thay vì int  
for (int i = 1; i <= n; i++) {  
    tong += i;  
}
```

Python

```
# Python tự xử lý số lớn, không cần lo  
tong = 0  
for i in range(1, n + 1):  
    tong += i
```